

Day 53 -- Line-by-Line Syntax Explanation

day53_residual_connections.py, segment by segment | MD Mutasim Billah

1. conv2d_forward_mc/backward_mc -- Padding AND Vectorization, Both New Today

forward: looping over kernel offsets instead of output pixels

```
for kh in range(kH): # loop over kernel row offsets (e.g. 3, not out_H which could be 40)
for kw in range(kW): # loop over kernel column offsets
X_slice = Xp[:, :, kh:kh+out_H, kw:kw+out_W] # every output position's pixel at THIS offset,
whole batch at once
Z += np.einsum("mchw,fc->mfhw", X_slice, W[:, :, kh, kw]) # contract over input channels,
broadcast over filters
```

Days 39-52 looped over every OUTPUT position ($C_{out} \times out_H \times out_W$ iterations -- e.g. $8 \times 36 \times 36 = 10,368$ for one conv). Today loops over kernel OFFSETS instead ($kH \times kW$ iterations -- e.g. $3 \times 3 = 9$), using `np.einsum` to handle every output position and every filter in one vectorized call per offset. Verified numerically identical to the old loop version (differences at $\sim 1e-15$ floating-point noise) and re-passes the same gradient check -- roughly 4x faster on this lesson's 7-conv-layer networks, which is what keeps a 150-epoch, two-network comparison practical.

the padding this lesson also needed, for the residual add to shape-match

```
if pad > 0: # only pad when asked to
Xp = np.pad(X, ((0,0),(0,0),(pad,pad),(pad,pad))) # zero-pad only height/width, not
batch/channel
...
```

A residual block needs $F(x)$ and x to have the SAME shape to add them. An unpadded 3×3 conv shrinks spatial size by 2; padding=1 exactly cancels that. `conv2d_backward_mc` re-pads X the same way internally, then crops the padding back off dX before returning it, so callers never see padded shapes.

2. plainblock_forward/backward vs. resblock_forward/backward

the ONLY structural difference between a plain and a residual block

```
# plain:
out = relu(z2) # no identity added
# residual:
out = relu(z2 + x) # add the block's OWN INPUT before the final relu
```

Everything else -- both conv layers, the ReLU between them, the cache shape -- is byte-identical between `plainblock_forward` and `resblock_forward`. This is a deliberate one-variable isolation, the same principle Day 50 used for dropout and Day 52 used for kernel shape.

resblock_backward: splitting gradient across two paths, then re-summing

```
dsum = dout * drelu(z2 + x) # undo the outer relu (applied to the SUM, not z2 alone)
dz2 = dsum # the sum's gradient flows unchanged into z2
dx_identity = dsum # AND unchanged into x directly -- the shortcut gradient path
...
dx = dx_from_conv1 + dx_identity # THE KEY LINE: combine both paths' contributions to dx
```

$d(z2+x)/dz2 = 1$ and $d(z2+x)/dx = 1$ -- the sum's gradient distributes UNCHANGED to both branches. $dx_identity$ is added to whatever the conv path (dx_from_conv1 , computed by backprop through conv2, relu, conv1 in the normal way) contributes -- this extra, unattenuated term is the entire residual-connection argument, expressed as one line of code.

3. demo_gradient_flow_by_depth() -- the New Vanishing-Gradient Diagnostic

measuring gradient norm by layer, at initialization, before any training

```
plain = PlainDeepConvNet(C=C, img_size=img_size, n_blocks=n_blocks, lr=0.15, seed=42)
resnet = ResNetStyleConvNet(C=C, img_size=img_size, n_blocks=n_blocks, lr=0.15, seed=42) #
SAME seed
plain.forward(Xb, training=True); plain.backward(yb_oh) # one forward+backward, populates
plain.grads
resnet.forward(Xb, training=True); resnet.backward(yb_oh) # same, for the residual network
np.linalg.norm(plain.grads["blocks"][i][2]) # Frobenius norm of one layer's weight gradient
```

Same seed on both constructors means both networks start from IDENTICAL initial weights -- confirmed directly via `np.allclose()` before comparing. Any difference in gradient norm is then attributable ONLY to whether a block adds its identity shortcut, not to any confound from different starting weights. This measures the CAUSE (gradient magnitude by depth) rather than Day 52's dead-fraction diagnostic, which measures a downstream SYMPTOM after training has already happened.

4. demo_fair_rematch() -- Calibration Sweep, Then the Full Comparison

a short sweep across a wider, lower lr range than Day 52 needed

```
candidate_lrs = [0.15, 0.05, 0.02, 0.008] # descending from stage 3a's failed rate to a
gentle one
for lr in candidate_lrs:
    p = PlainDeepConvNet(..., lr=lr, seed=42)
    train_with_history(p, ..., epochs=sweep_epochs, ...) # a SHORT 25-epoch budget, just to
    screen lrs
    ...
chosen_lr = max(sweep_results, key=lambda row: row[2])[0] # whichever lr gave ResNet its best
short-run accuracy
```

This lesson's network is deeper than Day 52's (7 conv layers vs. 4), so the sweep checks a wider, lower range. Each candidate trains for only 25 epochs -- enough to see whether a network escapes chance-level accuracy at all -- before committing to the full 150-epoch budget only once, at whichever lr looks most promising, saving significant compute versus running every candidate for the full budget.